

README

Docs Chain

Revision: 4 **Last modified:** 2026-05-31T12:00:00Z **Status:** Docs Chain Phases 1–3 plus the Phase-4 config loader + CLI **IMPLEMENTED** + tested (internal/hash + internal/graph + internal/adaptor + internal/orchestrator + internal/config + internal/state + internal/runner + cmd/docs_chain — `go test -race ./...` passes). The first downstream consumer (Herald) has wired a full 66-doc Markdown→HTML/PDF/DOCX corpus via `exec:` transforms and `verifys` exit 0 across it (see [docs/USE_CASE_CATALOGUE.md Appendix Z](#)). The `fsnotify` watch daemon, the generic bidirectional `md-to-sqlite / sqlite-to-md` DB builtins, the `colorize-html` (§11.4.23) builtin, and the comprehensive test suite (Phase 5 — real-binary subprocess `e2e` + watch full-automation + `scripts/e2e.sh`) are now **IMPLEMENTED** + tested (`go test -race ./...` passes; `scripts/e2e.sh` is GREEN, 20/0). Phase 6 (constitution-submodule distribution) + Phase 7 (ATMOSphere wiring) remain **PLANNED** — **OPERATOR-GATED** (remote-repo creation / cross-repo). This README and the docs under `docs/` mark each capability **IMPLEMENTED** vs **PLANNED** and do not claim working behaviour that is not yet built (§11.4.6). **Authority:** Operator mandate 2026-05-29 (Docs Chain initiative)

Docs Chain is a universal, Go-implemented, **bidirectional document-and-database dependency-propagation engine**. When any member of a registered chain changes — a Markdown source, an HTML/PDF export, or a SQLite database — Docs Chain detects the change by **content hash** (not mtime) and propagates it through every connected member in every declared direction, regenerating and re-exporting atomically, so no tracked artefact can fall out of sync.

It is the mechanical successor to the project's ad-hoc documentation-sync scripts. It ships as its own `vasic-digital` submodule and is consumed as a core part of the `HelixConstitution` submodule so any project adopts it out of the box and registers its own chains via per-context YAML.

Model in one line

Salsa-style content-hashed incremental recomputation over a DAG, with Kahn topological ordering, early-cutoff, declared-authority bidirectional sync edges, and atomic-rename + SQLite-transaction commit.

Implementation status

Phase	Scope	Status
0	Research + design + this documentation	Done (design artefacts)
1	Core DAG + content-hash engine	IMPLEMENTED + tested (internal/hash, internal/graph)
2	Node adapters + transforms	IMPLEMENTED + tested (internal/adapter)
3	Propagation orchestrator + atomicity (filesystem/SQLite)	IMPLEMENTED + tested (internal/orchestrator)
4	Config-driven multi-context + CLI + watch daemon	IMPLEMENTED + tested (internal/config, internal/state, internal/runner, cmd/docs_chain — sync/verify/doctor/graph/watch; fsnotify daemon proven by cmd/docs_chain/watch_test.go full-automation).
4b	Generic DB builtins + colorize-html	IMPLEMENTED + tested — bidirectional md-to-sqlite/sqlite-to-md (pure-Go modernc, row-level drift, byte-stable round-trip) + colorize-html (§11.4.23 cell-color matrix).
5	Comprehensive test suite (real-binary e2e + full-automation)	IMPLEMENTED — cmd/docs_chain/e2e_test.go (subprocess across every subcommand + exit-code contract), watch_test.go (daemon), internal/runner/sqlite_runner_test.go, scripts/e2e.sh (20/0 GREEN, captured transcript under qa-results/docs_chain/).
6	Constitution-submodule integration + repo creation	PLANNED — OPERATOR-GATED
7	ATMOSphere wiring + retire ad-hoc scripts	PLANNED

What is IMPLEMENTED today (go test ./... passes)

- **internal/hash** — Hasher interface + ByteContentHasher (LF normalization, trailing-whitespace strip, single trailing newline) so semantically-equivalent inputs collide by design; plus the sorted member-list fingerprint for roster/corpus sidecars (§11.4.86). Change detection is by **content hash, never mtime**.
- **internal/graph** — the DAG (Node/Edge/Graph), Validate (cycle detection → CycleError), TopoOrder (deterministic Kahn ordering), and the recompute engine: Recompute (early-cutoff — unchanged inputs skip transform), ResolveSync (declared-authority bidirectional sync edges with ConflictError), and CommitHashes. An in-memory Store (MemStore) backs the unit tests.
- **internal/adaptor** (Phase 2) — node-content adapters backing real stores: a FileAdapter for markdown (and the on-disk html/pdf/docx outputs) with per-file atomic temp-then-rename writes; DERIVED transforms that shell out to **pandoc** (md→html, md→docx) and **weasyprint** (html→pdf) — when a tool is absent they return a typed ToolAbsentError (IsToolAbsent) and never fake success; a SQLiteAdapter (pure-Go modernc.org/sqlite, no cgo) whose hashed content is a **canonical row dump** from a deterministic ORDER BY query — NOT the raw .db page bytes — so identical row sets collide regardless of insert order; and a FileStore implementing the Phase-1 graph.Store interface on top of these adapters, so graph.Recompute runs unmodified against real files and databases.
- **internal/orchestrator** (Phase 3) — Run ties graph.Recompute to the FileStore with three guarantees: **atomicity** (regenerated outputs are staged in-memory and written only after the whole run succeeds; any transform error rolls back with zero partial writes — composes with §9.2), **cycle-guard** (refuses to run when graph.Validate reports a CycleError, writing nothing), and **sync-conflict** surfacing (ConflictError on a both-dirty sync pair, writing nothing). It returns the RecomputeResult plus a committed / rolled-back / in-sync / conflict / cycle Status. A staged intermediate feeds the next transform, so multi-level chains (md→html→pdf) propagate in one pass.
- **internal/config** (Phase 4) — the per-context YAML loader + validator (Load/LoadDir/Parse/Validate/BuildGraph). Parses .docs_chain/contexts/<name>.yaml into the node/edge/transform model, accepts the from: <id> / from: [<id>, ...] multi-input union, rejects unknown fields, and enforces every CONFIG_SCHEMA §8 rule (empty context, dangling node/transform refs, builtin-xor-exec, fingerprint members, sync authority ∈ {a,b}, derive-from acyclicity) with loud *ConfigErrors.
- **internal/state** (Phase 4) — state.json content-hash baseline (<root>/.docs_chain/state.json, gitignored/regenerable per §11.4.77) with atomic temp-then-rename save; a missing file is a clean cold start.
- **internal/runner** (Phase 4) — wires a loaded context to the Phase 1–3 engine: registers a Phase-2 adaptor per node, binds each config transform to a real graph.Transform (builtins pandoc-html / weasyprint-pdf / pandoc-docx / members-fingerprint; exec: transforms stage input/output temp files per CONFIG_SCHEMA §5.2 and shell to the consumer's binary), hydrates the hash

baseline from state, runs the orchestrator (sync), and performs the read-only sink-side drift check (verify) into a temp output so it never mutates a live artefact.

- **cmd/docs_chain** (Phase 4) — the consumer-facing CLI with the documented exit-code contract (0 in-sync/applied · 1 error · 2 conflict · 3 transform-fail · 4 cycle/config-error). Subcommands: doctor (validate + tool-availability, no writes), sync (atomic propagate + state.json update + qa-results/docs_chain/<run-id>/ evidence), verify (read-only CI gate, non-zero on drift), graph (topo order + edges). Tool-absent runs roll back honestly (no fake output) and the message says so.

What is PLANNED (NOT yet functional — do not assume working behaviour)

Only the **distribution** / **wiring** phases remain, and they are OPERATOR-GATED (an agent must not create remotes or add submodule pointers — §11.4.66):

- **Phase 6** — constitution-submodule integration + repo creation: publish vasic-digital/docs_chain and add it as a submodule of the constitution so consumers inherit the engine by-reference at the constitution-exposed path.
- **Phase 7** — register the ATMOSphere contexts + retire that project's remaining ad-hoc doc-sync scripts.

The engine itself is feature-complete: sync/verify/doctor/graph/watch, all builtins (pandoc-html/weasyprint-pdf/pandoc-docx/members-fingerprint/md-to-sqlite/sqlite-to-md/colorize-html), and exec: transforms are IMPLEMENTED + tested. docs/ pages describe Phases 6–7's DESIGNED contract.

Documentation

Document	Purpose
docs/ARCHITECTURE.md	In-depth system architecture — DAG model, content-hash detection, early-cutoff, Kahn propagation, bidirectional sync authority + conflicts, atomicity/crash-safety, SQLite integration. Mermaid + ASCII diagrams.
docs/USER_GUIDE.md	End-to-end adoption guide — prerequisites, build, init .docs_chain/, define a context, run sync, the watch daemon, conflict resolution, CI integration.
docs/CONFIG_SCHEMA.md	Formal per-context YAML reference — every field, type, allowed values, defaults, validation rules; annotated derive-from + sync examples.
docs/USE_CASE_CATALOGUE.md	Living registry of 8 ready-to-use chain recipes (issues, fixed, status,

Document	Purpose
<u>docs/CONSTITUTION_INTEGRATION.md</u>	roster/corpus, changelog, README doc-link, universal markdown-export, CONTINUATION), plus Appendix Z — a worked, verify-green consumer example (Herald) showing the relative-asset-stable <code>exec:</code> export pattern. How the constitution submodule makes Docs Chain available to every consuming project — inheritance model, config discovery, the §11.4.x anchors it satisfies, what a project gets for free vs must register.

Quick start (CLI — Phase 4)

```
git clone git@github.com:vasic-digital/docs_chain.git
cd docs_chain
go test -race ./... # whole suite - all green
go build -o ./docs_chain ./cmd/docs_chain

# In a consuming project root that has .docs_chain/contexts/
#   <name>.yaml:
./docs_chain doctor --root . guide # validate (no writes)
./docs_chain sync --root . guide
# propagate atomically, update state.json
./docs_chain verify --root . guide # read-only CI gate; non-zero
# on drift
./docs_chain graph --root . guide # print topo order + edges
```

Exit codes: 0 in-sync/applied · 1 error · 2 conflict (both sides of a sync edge dirty) · 3 transform-fail (rolled back, no live changes) · 4 cycle/config-error. A sync run records evidence under `qa-results/docs_chain/<run-id>/`; the content-hash baseline lives in `<root>/docs_chain/state.json` (gitignored, regenerable). When `pandoc` / `weasyprint` is absent the run rolls back honestly and says so — it never fakes an export.

Library API (Phase 1 core engine)

The core is also usable programmatically. Minimal use:

```
import (
    "digital.vasic.docs_chain/internal/graph"
    "digital.vasic.docs_chain/internal/hash"
)

g := graph.New()
// AddNode for each member, AddEdge for derive-from / sync
// relationships,
```

```
// then g.Validate() (cycle check) and g.TopoOrder() (Kahn
    ordering).
res, err := g.Recompute(store, hash.NewByteContentHasher(),
    transforms)
// res reports which targets changed (early-cutoff skips unchanged
    inputs);
// g.CommitHashes(res) persists the new content hashes.
```

See [docs/USER_GUIDE.md](#) for the full DESIGNED workflow and [docs/CONFIG_SCHEMA.md](#) for the per-context YAML contract.

License

Part of the vasic-digital toolkit. Distributed under the same terms as the surrounding vasic-digital / HelixConstitution submodule family.

Design provenance

The authoritative Phase-0 design (DESIGN / RESEARCH / PLAN) lives in the consuming project's research tree at [docs/research/docs_chain/](#) and is mirrored into the consuming project, not into this standalone repo. The [docs/](#) pages here are the self-contained, comprehensive specification of the DESIGNED system.